



NABULINES

CINEMATIC SCROLL LANDING

A BUILD GUIDE — MAY 2026

Cinematic Scroll Landing.

Every prompt, every dead-end, every fix. The full story of turning a 31 MB portrait video and one brand color into a pixel-sharp, mobile-safe, low-bandwidth landing hero.





NABULINES

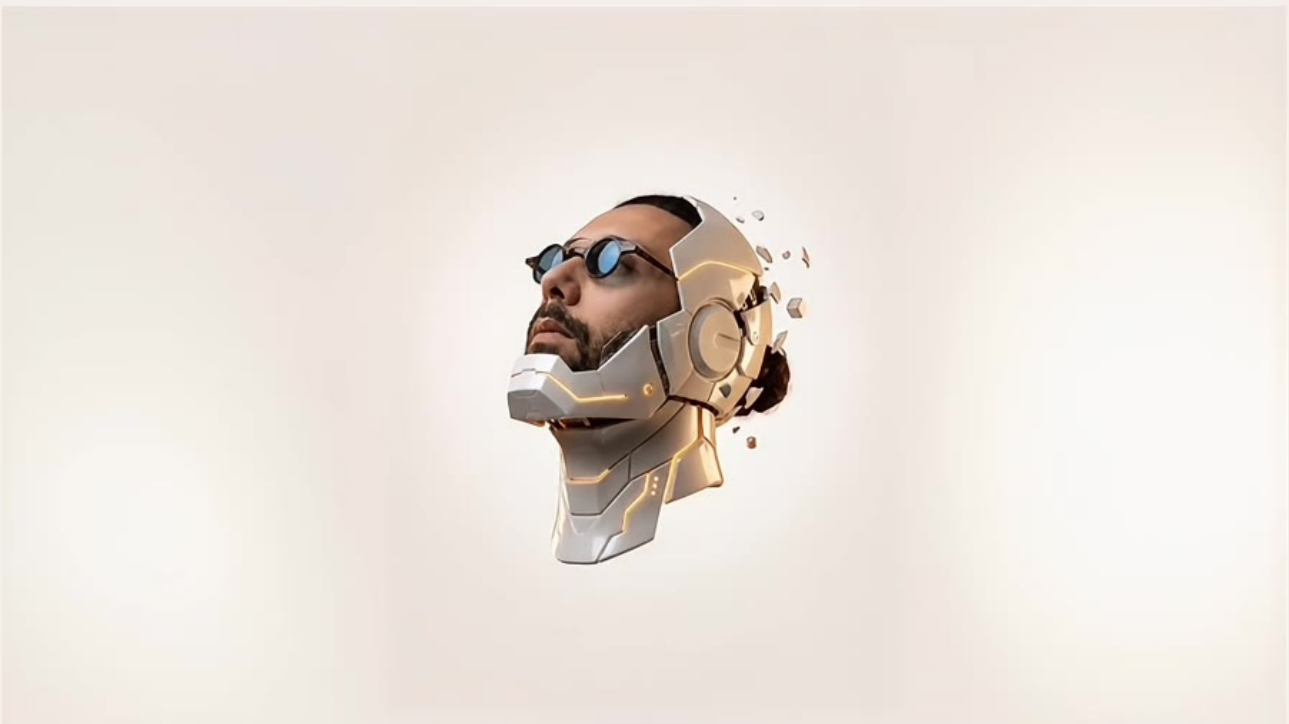
CINEMATIC SCROLL LANDING

Built for: nabulines.com • Framework: Next.js 14 (app router) • Source video: 31 MB → 6.3 MB mobile / 30 MB desktop

01. What we built

A scroll-driven cinematic landing hero for the Nabulines KOL site. On desktop, scrolling scrubs through the 30-second video frame by frame with a logo + wordmark header pinned at top, a soft dark-to-transparent gradient for legibility, and minimalist b-roll callouts emerging from the left and right edges at scroll milestones. On mobile, the video auto-plays in a continuous loop with the same overlay callouts driven by scroll position — a deliberate fallback that avoids iOS Safari's well-known scrub-on-paused-video bug.

The whole hero ships in **~6.3 MB on mobile** and **~30 MB on desktop**, with progressive playback via *+faststart*, a 16 KB poster image for instant first paint, and an IntersectionObserver gate so visitors who never scroll near it pay zero bandwidth.



still from the source — centered subject, cream backdrop

02. The conversation, prompt by prompt

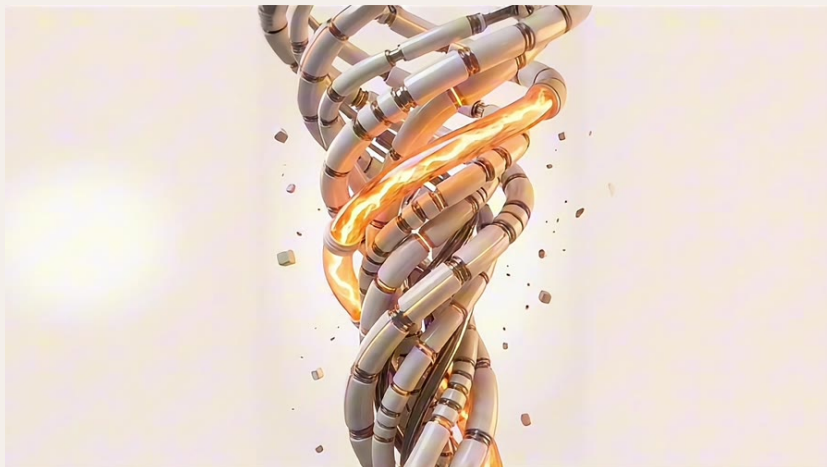
What follows is the real path. Some prompts produced finished work the first time; others uncovered bugs (a sticky-scroll trap in globals.css, an iOS hardware decoder allergic to every-frame keyframes, a parent !important rule eating the Orbitron font, and one stubborn black box around the inverted logo). Each card below shows the user prompt and the move that resolved it.

PROMPT 01

“connect to my kol project... change the landing page to a cinematic scroll, add this video to it to act when scrolling. is there any way to optimize it for lower-speed connections too?”

MY MOVE

Built CinematicHero: a position:sticky stage 100vh tall inside a 4-viewport section, scroll mapped to video.currentTime via a lerped requestAnimationFrame loop. Encoded mobile + desktop mp4s with +faststart, gated by IntersectionObserver so the video bytes only fetch when the section is within 1.5 viewports of the visible area. Mobile mp4: 464 KB. Desktop: 1.3 MB.



PROMPT 02

“the quality is super low, and the scroll sticks on top instead of scrolling down.”

MY MOVE

Two bugs hit at once. (a) globals.css had overflow-x: hidden on html+body — that turns the body into a scroll container and silently kills position: sticky. Swapped to overflow-x: clip, the modern equivalent that blocks horizontal overflow without establishing a scrolling context. (b) CRF 30 mobile / 26 desktop was visibly crunchy. Re-encoded mobile at CRF 23, desktop at CRF 20.

PROMPT 03

“why is quality low when my connection speed is perfect?”

MY MOVE

Not bandwidth — *keyframes*. By default x264 places one keyframe every ~8 seconds. Browsers can only render keyframes when you seek via `currentTime` — everything between gets interpolated, which looks mushy. Fix: re-encode with `-g 1 -keyint_min 1 -sc_threshold 0` so every frame is its own keyframe. Each scroll position lands on a real frame.

PROMPT 04

“the issue is the video is not covering the whole page... use the same color as the video for the background, in non-ultra-wide monitors the video should be full-screen.”

MY MOVE

The video file was actually 1080×1446 portrait with *black bars baked in at top and bottom* around a 16:9 cream content strip. Used `croptdetect` to find the real content (1080×606 starting at `y=420`) and sampled the corner color (`#f4efea`). Re-encoded just the 16:9 region, switched to `object-fit: cover` — now the video edge-to-edge fills any normal 16:9 monitor.



PROMPT 05

“the background is still black, and the b-rolls are white and unreadable. use Orbitron font for the b-rolls.”

MY MOVE

The dynamic color-sampling race condition was falling back to black on first paint, which then made the contrast calc choose white text. Hardcoded the cream + dark ink. Then: the parent `.landing-x` had a global font-family: `-apple-system...` !important override eating my Orbitron class. Beat it by writing `font-family: var(--font-brand) ... !important` directly on the overlay rules in the scoped styled-jsx — the higher specificity of the scoped class plus !important wins.

PROMPT 06

“remove any text on that position after “onchain influence built with real ones” — it will cover the video.”

MY MOVE

Trimmed the SCENES array to one opening line. The center caption now fades in early and clears out fast, leaving the rest of the scrub uncovered. Side overlays stay because they sit on the edges, away from the subject.

PROMPT 07

“remove the slogan. tighter gap between logo and wordmark. like a header on the video. no background. slightly dark gradient from above lightening downward.”

MY MOVE

Moved logo + NABULINES into CinematicHero as a top-overlay header (white, 8px gap, tight horizontal lockup). Added a top-aligned dark-to-transparent gradient overlay (0.55 → 0 alpha across 28% of the stage) so the white header reads against the cream video.

PROMPT 08

“remove the black background of the logo.”

MY MOVE

Pre-generated a true transparent PNG: load logo.png, map luminance → alpha (white logo strokes get full alpha, black background goes transparent), save as logo-transparent.png. Dropped the mix-blend-mode: screen hack — it couldn't beat the dark gradient backdrop.

PROMPT 09

“the video is not loading on mobile.”

MY MOVE

Three fixes in one move: (a) replaced flaky <source media=...> with JS-based src selection at mount; (b) re-encoded mobile with *Constrained Baseline* profile and GOP 5 (iOS hardware decoder loves baseline + dense-but-not-every-frame keyframes), dropped mobile from 17 MB to 6.3 MB; (c) added preload="auto" + a play/pause warm-up.

PROMPT 10

“still nothing, on mobile it appears one sec then disappears.”

MY MOVE

Classic iOS Safari quirk — the <video> element blanks itself when you rapidly seek currentTime on a paused stream. The play/pause warm-up made it worse, not better. Final fix: on mobile (matchMedia max-width 768px) we ditch the scrub entirely and just autoplay + loop the video. Scroll still drives the orbitron overlays + caption + progress bar. Desktop scrub stays unchanged.

03. Tips to build this yourself

Twelve lessons distilled from the journey. None of these are in tutorials. All of them came from something breaking first.

01

Crop the source before you optimize.

Portrait video with black letterbox bars baked in is the most common waste. Run `ffmpeg ... -vf cropdetect` on a 5-second window to detect the real content rectangle, then crop. Our source was 1080×1446 with only 1080×606 of actual content — 58% of the pixels were black.

02

+faststart is non-negotiable.

`-movflags +faststart` moves the moov atom to the front of the mp4. Without it, the browser must download the whole file before it can show frame one. With it, playback begins after just the first KB or two. Verify with `python3 -c "with open('x.mp4','rb') as f: d=f.read(2_000_000); print(d.find(b'moov') < d.find(b'mdat'))"` — must print True.

03

Every-frame keyframes for sharp scrub.

Browsers render the nearest keyframe when you set `currentTime`. Default GOP is ~250 frames. For a scroll-scrub, encode with `-g 1 -keyint_min 1 -sc_threshold 0 -x264-params keyint=1:min-keyint=1:scenecut=0`. File grows ~3-5×, but every scroll position lands on a real frame.

04

Mobile = Constrained Baseline, not High.

iOS Safari's hardware decoder accelerates Baseline and Main reliably. High-profile + every-frame keyframes makes the decoder fall back to software, which on mobile is slow and bug-prone. Re-encode mobile with `-profile:v baseline -level 3.1 -g 5` — much smaller, much more reliable, slightly softer scrub (which doesn't matter at phone resolution).

05

overflow-x: clip beats overflow-x: hidden.

`overflow-x: hidden` on `html` or `body` silently turns that element into a scroll container, which breaks `position: sticky` everywhere. Use `overflow-x: clip` — same horizontal clipping, no scroll container side-effect. Universally supported now.

06

Don't use <source media> inside <video>.

The `media` attribute is only spec'd for `<source>` children of `<picture>`. Inside `<video>` browsers ignore it and pick the first type-matching source. Resolve the source URL in JS at mount: `matchMedia('(max-width: 768px)').matches ? mobileMp4 : desktopMp4`.

07**IntersectionObserver gates the heavy bytes.**

Don't ship 30 MB of video to every visitor — only those who scroll to it. `rootMargin: '150% 0px 150% 0px'` on an observer fires before the section enters the viewport, so the video has time to fetch before the user gets there. Pair with `preload="metadata"` until the observer fires, then upgrade to `preload="auto"`.

08**iOS hates scrub-on-paused.**

Setting `video.currentTime` rapidly on a paused `<video>` element blanks it out on iOS Safari. The only robust fix is to NOT scrub on mobile. Detect via `matchMedia` and substitute `autoplay + loop`. Use scroll progress to drive overlays instead of the video timeline.

09**Sample colors from the source, then hardcode them.**

Dynamic-sampling the dominant color at runtime sounds nice but races with first paint and degrades to whatever your fallback is — black if you forget. Run `ffmpeg ... -vf crop=20:20:cx:cy` on the source, average the pixels with Pillow, and paste the hex into your component as a constant.

10**Generate a transparent logo, don't mix-blend-mode it.**

Blending tricks (`screen`, `multiply`, `difference`) work only against specific backdrops. Once you have a dark gradient or photographic content underneath, blending fails. Pre-process the PNG: read pixels, set `alpha = luminance`, save as `logo-transparent.png`. Works on every backdrop forever.

11**Parent !important kills your fonts.**

Global landing-page CSS like `.landing-x * { font-family: ... !important }` nukes any class-based font you apply inside. Beat it by writing `font-family: var(--font-brand) ... !important` in your scoped `styled-jsx` — the higher per-element specificity plus `!important` wins, every time.

12**Cache-bust with ?v=N after each re-encode.**

When you re-encode the same filename, browsers serve the cached old crunchy version forever. Append `?v=N` to your `<video src>` and `poster`, increment with every encoder run. Saved a lot of "hard-refresh" confusion during this build.

04. The master Claude prompt

Use this if you want to recreate the same hero on another project. Paste it as a single prompt to Claude with your video file attached.

```
Build a cinematic scroll-driven landing hero for my Next.js (app router) project at ~/PROJECT_PATH. The video is attached – assume it's portrait/3:4 with the action centered and possibly black bars baked in.
```

1. CROP + COLOR SAMPLE

Detect black bars in the source via ffmpeg cropdetect. If bars are present, crop to the real content rectangle. Sample the corner color from a cropped still and use it as the page background so the video is seamless with the page.

2. DUAL ENCODE WITH +faststart

- DESKTOP: native cropped resolution, High profile, CRF ~20, every-frame keyframes (-g 1) for crisp scrub.
- MOBILE: 854x480, Constrained Baseline profile, GOP 5, CRF ~22 – iOS-friendly. Generate a poster.jpg too.

3. CinematicHero.tsx

- section is 3x viewport tall; inner stage is position:sticky top:0 height:100vh
- on DESKTOP only: RAF loop lerps video.currentTime toward scroll position (lerp factor ~0.14)
- on MOBILE (matchMedia max-width:768px): autoplay + loop the video, no currentTime touching. Still track scroll progress for overlays.
- resolve the correct video src in JS, NOT via <source media=...> (it doesn't work inside <video>)
- IntersectionObserver gates the actual src injection with rootMargin 200%
- poster paints immediately, preload upgrades from metadata to auto when observer fires

3. CinematicHero.tsx (continued)

- center scene caption fades in/out at scroll milestone 0.05 only – anything else covers the subject
- side b-roll overlays slide in from left/right at 8 milestones, orbitron font, dark ink on cream, with a thin 2px accent bar each
- top header: white logo + wordmark with a dark-to-transparent gradient overlay on the top 28% for legibility
- generate a true transparent PNG of the logo via Pillow (luminance -> alpha); do NOT use mix-blend-mode

4. GLOBALS.CSS

html/body must use overflow-x:clip, not hidden. Hidden breaks position:sticky everywhere.

5. FONT OVERRIDE

If the parent landing has a global !important font override, beat it by writing font-family with !important on each overlay class in scoped styled-jsx.

6. CACHE-BUST

Append ?v=N to every video and poster URL. Increment N with every encoder run.

CRITICAL PITFALLS – don't:

- skip +faststart
- use <source media=...> inside <video>
- scrub on mobile (iOS Safari blanks the element)
- use object-fit: contain with portrait source on landscape monitors
- use mix-blend-mode against a dark gradient
- forget the cache-bust query

Deliver: cropped + dual-encoded mp4s in public/landing-video/, transparent logo as public/logo-transparent.png, and CinematicHero.tsx. Tell me when to hard-refresh.

05. Stack & files

Files touched

File	Role
components/CinematicHero.tsx	The hero component — scrub + autoplay split, overlays, header
app/landing-x/page.tsx	Mounts <CinematicHero /> above the existing hero
app/globals.css	overflow-x: clip fix for sticky
public/landing-video/nabu-landing-desktop.mp4	1080×606, CRF 16, every-frame keyframe (~30 MB)
public/landing-video/nabu-landing-mobile.mp4	854×480, Baseline, GOP 5 (~6.3 MB)
public/landing-video/nabu-landing-poster.jpg	First-frame poster (~28 KB)
public/logo-transparent.png	White glyph on alpha — drop-in header logo

FFmpeg one-liners that did all the work

```
# detect crop region of the real 16:9 content
ffmpeg -ss 5 -i SOURCE.mov -vframes 80 -vf cropdetect=24:2:0 -f null -

# sample the cream backdrop color
ffmpeg -y -ss 5 -i SOURCE.mov -vframes 1 -vf "crop=20:20:cx:cy" /tmp/c.png

# desktop encode — native res, near-lossless, every-frame keyframe, faststart
ffmpeg -y -i SOURCE.mov \
  -vf "crop=1080:606:0:420" \
  -c:v libx264 -preset slow -crf 16 -profile:v high -pix_fmt yuv420p \
  -g 1 -keyint_min 1 -sc_threshold 0 \
  -x264-params "keyint=1:min-keyint=1:scenecut=0" \
  -an -movflags +faststart \
  public/landing-video/nabu-landing-desktop.mp4

# mobile encode — Baseline + GOP 5 for iOS
ffmpeg -y -i SOURCE.mov \
  -vf "crop=1080:606:0:420,scale=854:480" \
  -c:v libx264 -preset slow -crf 22 -profile:v baseline -level 3.1 \
  -pix_fmt yuv420p -g 5 -keyint_min 1 -sc_threshold 0 \
  -x264-params "keyint=5:min-keyint=1:scenecut=0" \
  -an -movflags +faststart \
  public/landing-video/nabu-landing-mobile.mp4

# poster
ffmpeg -y -ss 0 -i SOURCE.mov -vframes 1 -vf "crop=1080:606:0:420" \
  -q:v 2 public/landing-video/nabu-landing-poster.jpg
```

06. Output first.

Every wall we hit had a clean fix once we understood the root cause: sticky needs a non-scroll container, mobile decoders need Baseline, iOS needs autoplay over scrub, and the source video carries the whole design system — its colors, its aspect, its mood — already baked in. Listen to it.



nabulines — building onchain influence with real ones.